

Ranking Degeneracy: A Learned Wait-Time Model Schedules No Better Than Sorting by Requested Size

Rakshit Rameshbabu
Vellore Institute of Technology, Chennai

September 11, 2026

Abstract

A common recipe in “ML for systems” is to train a regressor that predicts job wait time from cluster-state features and then use its predictions to order the queue. We report that this recipe is *structurally degenerate*. At any single dispatch instant every queued job observes the same cluster, so in the feature vector only the job’s own attributes differ, and in the standard cluster-state feature set every such attribute is a deterministic function of the job’s requested size. The learned score is therefore a function of requested size alone, and the ordering it induces is a per-instant lookup table from size to priority: the cluster-state features shift all scores equally and cannot reorder anything. We verify this over 45,432 dispatch instants in three settings — 41,786 replayed from two Parallel Workloads Archive traces and 3,646 from a synthetic generator — and find zero counterexamples — two jobs of equal size never receive different scores. The consequences are measurable. On the synthetic benchmark a one-line sort by requested size is statistically *equivalent* to the full XGBoost pipeline (paired TOST $p = 2.6 \times 10^{-16}$, difference CI $[+0.03, +0.23]$ ts against a ± 1.59 ts margin), and an MLP on the same features reproduces the size sort exactly. We then re-ask the whole scheduler comparison on two real traces, replacing the simulated user-estimate model with the estimates the traces actually contain. The synthetic 7.9% mean-wait gain over FCFS does not replicate: the ML scheduler is indistinguishable from FCFS on LANL CM-5 ($p = 0.48$) and any classical policy with a runtime estimate beats it on both machines. *As a second and independent contribution* — independent because it concerns the estimate-error

model rather than the ranking score — we measure real user-estimate error against the f -model that the backfill literature assumes, and find real error far more damaging to backfill than the over-estimate-only f -model predicts (+74% mean wait on LANL, where 36.3% of user estimates are *under-estimates* that the f -model cannot generate at all). We argue the failure is one of feature design rather than model class, state the condition a wait-time feature set must satisfy to be non-degenerate, and release the harness, traces, and seeds.

1 Introduction

Machine learning for cluster scheduling is an active area, and a recurring design is: (i) log cluster state, (ii) train a regressor to predict a job’s wait time, (iii) at each scheduling decision, score the queued jobs with the regressor and dispatch in ascending predicted wait. The intuition is that a model which has seen how the queue evolves can anticipate contention better than a fixed heuristic.

This paper reports that step (iii) does not do what it appears to do. The argument is elementary but, as far as we can tell, not stated in the literature we build on, and it invalidates the natural reading of any improvement measured this way.

The degeneracy. Fix a dispatch instant t with cluster state S_t and queue Q_t . The feature vector for job j splits into two groups: features of the machine (free processors, queue length, number of running jobs, fragmentation, queue pressure) and features of the job (requested size, and quantities

derived from it such as “does it fit now”). The first group is *identical for every job in Q_t* — it describes S_t , not j . So for two jobs $j_1, j_2 \in Q_t$ the predicted waits can differ only through the second group. In the 12-feature set used here (and in the 8-feature set reconstructible from a Standard Workload Format trace) every member of the second group is a deterministic function of the requested size g_j once S_t is fixed:

$$\begin{aligned} \text{can_fit_now} &= \mathbb{I}[\text{free}(S_t) \geq g_j] \\ \text{gpu_fit_ratio} &= \min(\text{free}(S_t)/g_j, 1) \\ \text{node_availability} &= |\{n : \text{free}_n(S_t) \geq g_j\}|/N \\ \text{queue_pressure} &= (\Sigma_{Q_t} - g_j)/(\text{free}(S_t) + 1). \end{aligned}$$

Hence the score satisfies

$$\hat{w}(j | S_t) = g_{S_t}(g_j)$$

for some function g_{S_t} determined entirely by the state, and the induced queue order is the order of g_{S_t} over the sizes present in Q_t . The model is, at each instant, a *lookup table from requested size to priority*. The remaining seven features describe only the cluster — they select *which* table is used — but they cannot distinguish two co-queued jobs, which is the only thing a ranking consumes. (`queue_pressure` does differ between co-queued jobs, but only through the $-g_j$ term: it is itself a function of size given the state, so it cannot break the degeneracy.)

Why this matters. If the argument holds, then a reported improvement of an ML wait-time scheduler over FCFS is evidence about size-based ordering, not about learning. The correct control is not FIFO; it is *sorting by requested size*, which needs no dataset, no training, no inference, no SHAP explanation, and no drift monitor. To our knowledge that control is absent from the ML-scheduling evaluations we are aware of, including our own earlier versions of this work.

A second contribution. The trace-driven re-evaluation required us to replace the simulated user-estimate model with the estimates the traces actually contain, and that substitution turned out to carry a finding of its own, independent of the degeneracy. The standard f -model, $\hat{r} = r \cdot f$ with

$f \sim U(1, C)$, can only over-estimate. Real estimates on LANL CM-5 are under-estimates 36.3% of the time, and under-estimates break the very mechanism EASY backfilling relies on: EASY with real estimates costs +74% mean wait against perfect estimates on LANL, where the f -model predicts near-insensitivity. This is a statement about how backfill should be evaluated, not about ML, and we report it as a second contribution (§7) rather than as a by-product of the first.

Contributions.

1. **The degeneracy, stated and measured** (§4). Over 45,432 dispatch instants across a synthetic generator and two real traces, two equally-sized co-queued jobs never receive different scores (0 violations). Seven of the twelve features vary across the queue in 0.0% of instants, and every feature that does vary is a deterministic function of the requested size. A nine-job queue receives only 2.3–3.1 distinct priority levels.
2. **An equivalence result, not a null result** (§5). Using paired TOST rather than a failure to reject, sorting by requested size is statistically equivalent to the XGBoost scheduler on the synthetic benchmark ($p_{\text{TOST}} = 2.6 \times 10^{-16}$) and on SDSC SP2 ($p_{\text{TOST}} = 1.8 \times 10^{-12}$). An MLP over the same features reproduces the size sort *exactly* on all 20 paired runs.
3. **A trace-driven re-evaluation** (§6). Replaying LANL CM-5 and SDSC SP2 through twelve policies, with the real user runtime estimates the traces contain, the synthetic 7.9% gain over FCFS does not replicate, and every policy with access to a runtime estimate beats the ML scheduler on both machines.
4. **Second contribution: real estimate error breaks backfill far more than the f -model suggests** (§7). Real estimates are $6.9\times$ the runtime at the median on SDSC but only $1.5\times$ on LANL, where 36% are *under*-estimates — a case the standard over-estimate-only f -model cannot produce.

Backfill costs +74% mean wait on LANL under real estimates versus perfect ones, against near-zero sensitivity under the f -model.

5. **A non-degeneracy condition and a reproducible harness** (§8). We state what a feature set must contain to make learning matter for ranking, and release code, traces, and seeds.

2 Related Work

Predicting queue wait time. Wait-time prediction for batch queues is long-standing: Downey’s log-uniform lifetime models, template-based prediction from historical similar jobs, and the QBETS/Karnak line of quantile-based bound predictors all predate modern ML pipelines [1, 5]. The recent line is explicitly ML-based. Park [19] predicts queue wait on a national supercomputer; Lovell et al. [21] use a hierarchical deep model for the same task; Ramachandran et al. [22] combine regressors with metaheuristic hyperparameter search; and Barsanti and Sodan [23] use predicted wait to drive adaptive resource allocation. Our contribution is not a better predictor. It is an observation about what happens when any such predictor is used as a *ranking* score, and it applies to this whole line: the relevant question for each is whether the feature set contains a per-job attribute that is not a function of requested size given the cluster state (§8).

Why reported gains over FIFO and this result are compatible. Many ML-for-systems papers report real gains against a first-in-first-out baseline, and nothing here contradicts them. Our claim is narrow in three ways, and a reported gain can sit outside all three. First, it concerns *ranking*: a learned component that drives *admission*, *placement*, or *backfill* selection is choosing among actions the queue order does not determine, and the degeneracy argument — which is about separating two co-queued jobs — says nothing about it. The deep-RL policies of Mao et al. [8, 9] are of this kind, as is the learned-index line in databases [32], where the learned object replaces a structure rather than a comparator. Second, it concerns a *feature family*: a feature set containing a genuine

per-job attribute — a runtime estimate, an application signature, a model-level descriptor as in Wang et al. [24] — is non-degenerate by Proposition 2, and a gain measured with such a feature set is a gain from that attribute. Third, it concerns the *control*: our result says the correct comparison for a degenerate feature set is sorting by size, not FIFO, and a gain over FIFO is simply silent about whether the size sort would have delivered it. So the claim compatible with both literatures is not “ML does not help scheduling” but “a gain over FIFO does not localise the mechanism”. Marcus et al. [33] make the corresponding point for learned indexes by benchmarking them against carefully tuned classical structures rather than against naive ones.

Learning to rank. Because the output we criticise is an ordering, the learning-to-rank literature [30, 31] is the natural place to look for an escape: replace the pointwise regression objective with a pairwise or listwise one that optimises the ordering directly. It does not escape. Proposition 1 constrains the feature map, not the loss, and we verified this empirically: an XGBoost ranker trained with `rank:pairwise` and one trained with `rank:ndcg` over the same features both produce *zero* counterexamples on both traces, as do monotone reshaping of the pointwise score (§9). A pairwise objective learns which of two jobs should go first; when the two jobs present identical feature vectors, there is nothing for it to learn from. This is worth stating because “we used a ranking loss” is the most plausible objection to the result, and it is measurably not a fix.

Backfilling and user runtime estimates. Backfilling [2] is the dominant production technique. EASY [6] reserves capacity for the queue head and backfills jobs that either finish before the reservation or fit in the surplus beyond the head’s requirement. Conservative backfill reserves for every queued job. Variants refine which job to pick and how much slack to allow: lookahead [12], slack-based priority [14], adaptive reservation [13], probabilistic backfilling on predicted-runtime distributions [11], and deviation backfilling [16]. All depend on user runtime estimates, and our §7

is in direct dialogue with two papers in particular. Mu’alem and Feitelson [6] define the two-condition EASY rule we implement and introduce the f -model, $\hat{r} = r \cdot f$ with $f \sim U(1, C)$, as the standard way to inject estimate error into a backfill evaluation. Our point is that this model is not merely imprecise but *structurally* unable to express the failure mode we measure: $f \geq 1$ means $\hat{r} \geq r$ always, so an f -model evaluation contains no under-estimates by construction, while 36.3% of LANL’s real estimates are under-estimates. An under-estimate is precisely what invalidates EASY’s reservation — a job admitted on the promise that it will finish before the reservation, and does not, delays the queue head — so the one error direction the model omits is the one that breaks the mechanism. Tsafir et al. [7] model user estimates in detail and, with Mu’alem and Feitelson, report the counter-intuitive finding that over-estimation can *help* backfill by making the scheduler conservative; we do not dispute that, and it is consistent with SDSC, where estimates are $6.9\times$ runtime at the median and EASY loses only 6.2% against perfect estimates. Our claim is that this benign regime is not universal: LANL, roughly calibrated at the median ($1.5\times$) but with a third of estimates too low, costs EASY $+74\%$. The literature’s own methodological warning applies to itself here — Feitelson [15] shows how strongly backfilling conclusions depend on exactly such evaluation choices — and we read our result as an instance of that warning rather than as a contradiction of [6, 7]. Tsafir et al.’s later move to system-generated predictions [10] is the constructive response, and it sidesteps the f -model question entirely by not using user estimates.

Replacing user estimates with predictions.

The closest prior work replaces the user’s estimate with a system-generated one. Tsafir et al. [10] predict runtimes from user history and backfill on the prediction; Minh and Wolters [17] predict application runtimes from historical data for the same purpose; Park and Hong [20] improve runtime prediction by conditioning on application type. Yuan et al. [18] note that naively feeding predictions into EASY causes reservation violations and add fairness guarantees to prevent it. This line differs from ours in the quantity pre-

dicted: a runtime estimate is a genuine per-job attribute, so ordering by it is *not* degenerate — which is precisely why SJF_USEREST beats the wait-time model in §6, and why a runtime estimate is our first suggested fix.

Learning-based schedulers. Deep-RL approaches [8, 9] learn a policy end-to-end rather than fitting a regressor and sorting by it, and so are not subject to the degeneracy as stated — though the same question (what is the ML-free control?) applies to them. Wang et al. [24] schedule ML cluster jobs from model-level features, an example of a feature set that *does* carry per-job information beyond size. Hovestadt et al. [25] frame the underlying choice as queueing versus planning.

Workloads and metrics. Our synthetic substrate is a simple generator, not a validated workload model; Lublin and Feitelson [26], Downey and Feitelson [27], and Feitelson and Shmueli [28] give the principled alternatives, and §9 treats this as a limitation. Zhang et al. [29] report cross-system variation in job characteristics of the kind that produces our LANL-versus-SDSC divergence. Bounded slowdown [4] and Jain’s index [3] are the standard evaluation metrics; we report both, having found that mean wait alone hides the effect on short jobs. Goponenko et al. [40] argue that these metrics correlate poorly with packing and fairness objectives, which is consistent with the metric-dependent verdicts we obtain in §5.

Negative results, weak baselines, and reproducibility.

The justification for publishing this at all comes from a literature about how empirical progress is measured. Armstrong et al. [34] showed that a decade of reported retrieval improvements largely failed to accumulate because each was measured against a weak baseline; Lin [35] and Dacrema et al. [36] found the same pattern in neural retrieval and neural recommendation, the latter reproducing eighteen published methods and finding most beaten by well-tuned non-neural baselines; Musgrave et al. [37] report the analogous result for deep metric learning; and Sculley et al. [38] argue that empirical rigour, not pace, is the binding constraint. Our finding is the

batch-scheduling instance of exactly this failure mode, with an unusually clean diagnosis: the appropriate strong baseline is not merely well-tuned, it is *derivable from the model’s own feature set*, and it is a one-line sort. We also follow this literature in method — equivalence testing rather than a failed difference test (§5), and reporting the split the accuracy number actually came from (§9) — and in artefact practice, since Collberg and Proebsting [39] document how rarely systems results can be re-run at all. Every number in this paper is produced by a single seeded pipeline whose per-step provenance is cited in each table and figure caption. A negative result is only worth reading if it can be checked.

3 Experimental Setup

3.1 Two evaluation substrates

Synthetic (for continuity). A discrete-time simulator: 8 nodes $\times$ 4 GPUs, 110 jobs arriving uniformly over the first 150 of 300 timesteps, sizes $U\{1, 8\}$, runtimes $U\{5, 20\}$. 20 paired runs (seeds $1000+i$, disjoint from the model’s training seeds $42+i$). This is the substrate all earlier versions of this work used, and we keep it only so the new controls can be compared against the previously published numbers.

Trace-driven (for validity). Two cleaned Parallel Workloads Archive traces: LANL CM-5 (1994, 1024 processors, 122,055 kept jobs) and SDSC SP2 (1998, 128 processors, 43,117 kept jobs). We reserve the first 60% of each trace’s time span for model training and carve 20 evenly spaced evaluation windows from the remainder, each 3 simulated days of warm-up (not measured) followed by 7 measured days. Mean offered load is 0.72 (SDSC) and 0.68 (LANL). Windows are evenly spaced, *not* selected by load, to avoid cherry-picking congestion.

The trace simulator is event-driven and exact to the second: state changes only at arrivals and completions, so there is no tick-quantisation bias. Because SWF records no per-node placement, we model capacity only; inventing node-level fragmentation would be fabricating data. §9 reports the resulting fidelity gap.

3.2 Policies

Twelve policies. FCFS (first-come first-served with unrestricted first-fit), FIFO_STRICT (a blocked head blocks the queue), SJF_ORACLE (true runtimes; unattainable), SJF_USEREST (the trace’s real requested time), HRRN_USEREST, SMALLEST_FIRST (sort by requested size — the degeneracy control), EASY_USEREST and EASY_ORACLE (canonical two-condition EASY), CONS_BF_USEREST, SRPT_ORACLE (preemptive, with a checkpoint penalty charged as waiting), PROACTIVE (the XGBoost wait-time model, re-trained per trace, no runtime information), and PROACTIVE_EST (the same model retrained with the user estimate as an extra feature — the control for “does ML add anything beyond the estimate?”).

The ML scheduler is given its best case throughout: a model retrained on the same machine’s own earlier data. Zero-shot transfer from the synthetic generator to these traces yields $R^2 \approx 0$ and is not a competitive configuration.

3.3 Metrics and statistics

We report mean wait, p95 wait, **mean bounded slowdown** $\max(\text{turnaround}/\max(r, \tau), 1)$ with $\tau = 60\text{s}$ [4], the Gini coefficient of waits, and utilisation. Wait is defined uniformly as turnaround minus true runtime, so preemptive requeue time and checkpoint overhead are charged to the preemptive policy rather than hidden.

Comparisons are paired (same workload, every policy) with Holm-adjusted t -tests and Wilcoxon signed-rank tests. Crucially, claims that two policies perform *the same* use paired TOST equivalence tests with a margin of 10% of the reference mean; a large p -value from a difference test is not evidence of equivalence, and treating it as such is how a degenerate policy survives review.

4 The Ranking Degeneracy, Measured

We instrumented every PROACTIVE ranking decision in real simulations (45,432 instants with ≥ 2 queued jobs) and recorded, per instant, which features differ across the queue and how the result-

ing order compares with ML-free orders. Table 1 summarises, and Figure 1 shows the per-feature variation directly.

	Synthetic	SDSC	LANL
Ranking instants	3,646	11,843	29,943
Features (total)	12	8	8
Features varying (mean)	2.62	2.87	2.69
Equal size, diff. score	0	0	0
Distinct scores / queue	2.81	3.09	2.28
Kendall τ vs. size	0.71	0.75	0.62
Order = smallest-first	80.3%	71.5%	77.6%
Order = arrival (FCFS)	19.7%	18.1%	27.5%
Size table monotone	63.1%	57.1%	56.9%

Table 1: The learned score never distinguishes two equally-sized co-queued jobs. Cluster-state features vary across the queue in 0.0% of instants. Source: `05_results/degeneracy/ranking_degeneracy.csv` and `05_results/degeneracy/feature_variation.csv`, produced by `python 04_scheduler/ranking_degeneracy.py` (step 9 of `run_all_experiments.sh`).

Three observations. First, the *zero* in row four is the claim: across 45,432 instants there is no counterexample, exactly as the argument in §1 requires. Second, of the twelve synthetic features only five ever differ between co-queued jobs (`job_gpu`, `gpu_fit_ratio`, `can_fit_now`, `node_availability`, `queue_pressure`) and all five are functions of the requested size; the remaining seven vary in 0.0% of instants and are therefore incapable of affecting any ranking. Third, the resolution is very coarse: a queue of ~ 9 jobs is partitioned into only 2.3–3.1 priority classes, and in 18–27% of instants *every* queued job receives the same score, so the policy silently reduces to FCFS.

The recovered size $\rightarrow$ priority table is monotone increasing in only 57–63% of instants. Where it is monotone the policy *is* smallest-job-first; where it is not, the model has learned a non-trivial size preference. We note explicitly that this fraction does *not* predict where the equivalence of §5 fires — SDSC (57.1%), where the size sort is certified equivalent, and LANL (56.9%), where it is not, are statistically indistinguishable on it. Degeneracy is the claim this section supports; the shape of the learned table is a description of it, not an explanation of the downstream scheduling difference. Figure 2 shows the recovered tables themselves.

4.1 The degeneracy, stated formally

Fix a dispatch instant t . Let S denote the cluster state at t — free GPUs per node, the running set, the queue contents, and any quantity derived from them — and let the queue be $Q = \{j_1, \dots, j_n\}$ with $g(j)$ the requested size of job j . A feature map assigns to each queued job $\varphi(j, S) = (\psi(j, S), \chi(S))$, where $\chi(S)$ collects the features that depend on the cluster alone and ψ those that depend on the job. A score function f gives $s(j) = f(\varphi(j, S))$, and the dispatcher ranks Q by s under a fixed tie-break.

Proposition 1 (ranking degeneracy). *Suppose every per-job feature factors through the requested size given the state: there is a map $\tilde{\psi}$ with $\psi(j, S) = \tilde{\psi}(g(j), S)$ for all $j \in Q$. Then there exists h_S with $s(j) = h_S(g(j))$ for all $j \in Q$.*

Proof. $\chi(S)$ does not depend on j , so at the fixed instant t it is a constant. Define $h_S(u) := f(\tilde{\psi}(u, S), \chi(S))$. Then for any $j \in Q$, $s(j) = f(\tilde{\psi}(g(j), S), \chi(S)) = h_S(g(j))$. $\square$

Corollary 1.1. Every feature in χ enters h_S as a constant argument at a fixed instant, so it cannot separate two queued jobs, *whatever* f is. No additivity or monotonicity assumption on f is needed: the feature is simply held fixed across the comparison. Seven of the twelve synthetic features are of this kind, and they are measured to vary across a queue in 0.0% of instants.

Corollary 1.2. If $g(i) = g(j)$ then $s(i) = s(j)$. This is the falsifiable form of the proposition, and it is what we count: zero counterexamples over 45,432 dispatch instants.

Corollary 1.3. The number of distinct scores at an instant is at most $|\{g(j) : j \in Q\}|$. The induced order is therefore measurable with respect to the partition of Q into size classes: the policy may order size classes and can do nothing within one. A queue of 8.5–10.2 jobs receives 2.28–3.09 distinct priority levels, and every queued job receives an identical score in 14.4–20.7% of instants, where the policy is exactly FCFS.

What the proposition does not say. It does *not* say the ranking is the ascending-size order. h_S may order the size classes arbitrarily; it may

A learned wait-time score cannot tell two queued jobs apart by anything but size

Every queued job sees the same cluster, so only per-job features can differ — and each of those is a function of requested size. Across 45,432 dispatch instants (3,646 synthetic, 41,786 replayed from the real SWF traces), two equally-sized jobs received different scores 0 times.

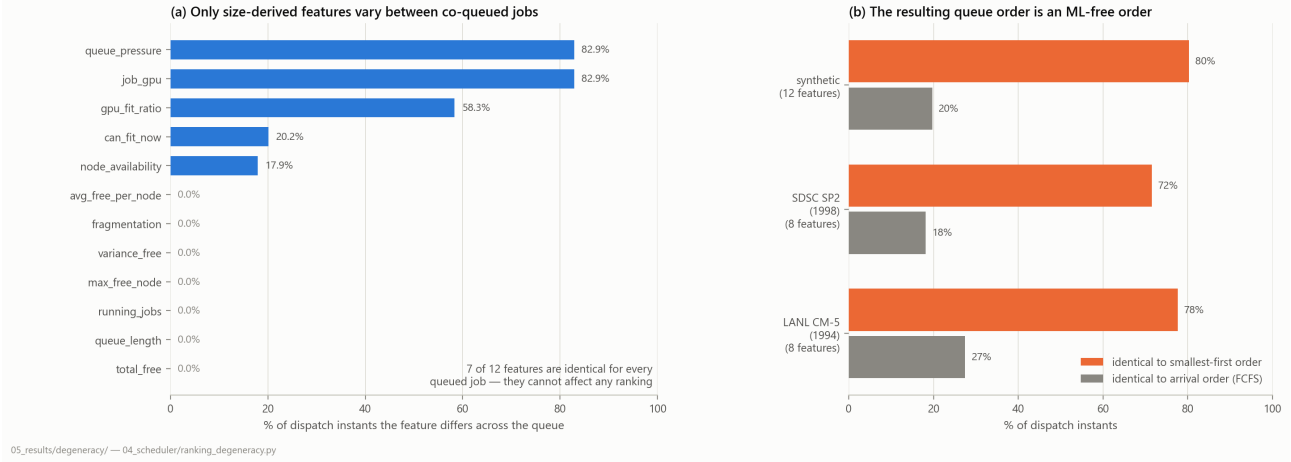


Figure 1: Feature variation in ranking decisions. What to look for: the bars for the seven cluster-state features sit flat on zero — they do not vary across a queue in *any* instant — while the only bars with any height belong to the job’s requested size and to the four quantities derived from it. The reader should also see that the same shape recurs in all three settings, so the degeneracy is not an artefact of one substrate. Source: `05_results/degeneracy/ranking_degeneracy.png`, plotted from `05_results/degeneracy/feature_variation.csv` and `05_results/degeneracy/ranking_degeneracy.csv`, produced by `python 04_scheduler/ranking_degeneracy.py` (step 9 of `run_all_experiments.sh`).

only not distinguish jobs within one. Our own measurements make the distinction concrete: the recovered size-to-priority table is monotone in just 57–63% of instants, and Figure 2 shows tables that are plainly functions of size without being increasing in it. This is why we establish equivalence to a size sort *statistically*, by TOST against SMALLEST-FIRST, rather than deducing it. Degeneracy is a structural fact; that the resulting policy performs like ascending-size order is a separate empirical claim.

The converse, and its limits. Proposition 2. *If some per-job feature does not factor through (g, S) — if $\psi(i, S) \neq \psi(j, S)$ is possible for two jobs with $g(i) = g(j)$ — then no such h_S need exist and Corollary 1.2 can fail. A feature set can produce a ranking finer than the size partition only if it contains such an attribute. Three limits bound what that buys. First, it is necessary but not sufficient: PROACTIVE-EST carries the user’s runtime estimate, is non-degenerate by Proposition 2, and still loses to plain SJF on both traces (§6). Second, the attribute must be infor-*

mative about wait, not merely distinct — a per-job nonce would break Corollary 1.2 while carrying no information. Third, the shared-state premise is a modelling choice rather than a law: a policy that scores a job when it *enters* the queue and caches the result violates it by construction, and whether such a policy schedules better is an empirical question in which staleness is a cost as well as a source of variation.

5 Sorting by Size Is Equivalent to the Model

Table 2 gives the paired TOST results.

On the synthetic benchmark the difference in mean wait is $+0.127$ ts with a 90% CI of $[+0.025, +0.228]$ against a ± 1.59 ts margin: the entire interval sits well inside the margin, so this is a positive finding of equivalence, not an underpowered null. The MLP baseline is a sharper version of the same point — NN and SMALLEST-FIRST produce *bit-identical* results on every metric of all 20 runs. Two different model classes over the

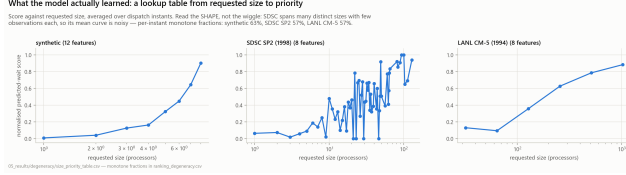


Figure 2: The recovered size→priority lookup table, per setting. What to look for: each curve is a *function* — one priority per requested size, which is the degeneracy — but it is not always an *increasing* function. The synthetic curve climbs monotonically from 0.008 at size 1 to 0.902 at size 8; the trace curves do not, dipping and rising instead (on SDSC, size 3 scores 0.017 against 0.072 at size 2). That is exactly the distinction Proposition 1 draws and its “what it does not say” note insists on: degeneracy forces a table, it does not force smallest-first. Across instants the table is monotone in only 63.1% (synthetic), 57.1% (SDSC) and 56.9% (LANL) of cases. Source: 05_results/degeneracy/size_priority_table.png, plotted from 05_results/degeneracy/size_priority_table.csv, produced by python 04_scheduler/ranking_degeneracy.py (step 9 of run_all_experiments.sh).

same features collapse to the same one-line sort. Figure 3 places both against the wider synthetic scheduler landscape.

On LANL the equivalence test does not fire, and the honest verdict there is *inconclusive* rather than “different”. The size sort is 14.4% worse on mean wait, but that difference does not survive Holm correction on the paired *t*-test ($p = 0.25$; the Wilcoxon test disagrees at $p = 0.015$), it shrinks to +7.3% and $p = 0.60$ on bounded slow-down, and TOST cannot certify equivalence either ($p = 0.77$). With 20 windows we can neither declare the two policies the same on LANL nor establish that they differ.

We initially attributed this to the learned size table being less monotone on LANL, and that explanation is wrong: the per-instant monotone fraction is 57.1% on SDSC — where equivalence *does* hold — against 56.9% on LANL. Monotonicity does not track where the equivalence fires, so we report the LANL result as underpowered rather than mechanistically explained. What is not in

Substrate	Metric	Δ	p_{TOST}	Eq.?
Synthetic	mean wait	+0.80%	2.6×10^{-16}	equiv.
Synthetic	bnd. slow.	+0.69%	3.8×10^{-18}	equiv.
SDSC SP2	mean wait	−0.05%	1.8×10^{-12}	equiv.
SDSC SP2	bnd. slow.	+2.7%	7.4×10^{-4}	equiv.
LANL CM-5	mean wait	+14.4%	0.77	no
LANL CM-5	bnd. slow.	+7.3%	0.30	no

Table 2: SMALLEST_FIRST versus PROACTIVE. Δ is the difference in the metric (positive = the size sort is worse). Equivalence margin 10% of the reference. “Eq.?” is the TOST verdict at that margin. Sources: 05_results/schedulers/multi_scheduler_equivalence.csv, produced by python 04_scheduler/multi_scheduler_benchmark.py (step 7 of run_all_experiments.sh), and 05_results/trace_schedulers/trace_scheduler_equivalence.csv produced by python 04_scheduler/trace_driven_benchmark.py (step 10).

question on either machine is the degeneracy itself (§1), and on LANL the ML scheduler also fails to beat plain FCFS ($p = 0.48$, §6).

6 Trace-Driven Re-Evaluation

Table 3 reports the twelve policies on both traces, and Figure 4 plots them.

The synthetic gain does not replicate. On the synthetic benchmark PROACTIVE beats FCFS by 7.4% (and by 7.9% in the older 40-run paired study). On SDSC it beats FCFS by 20.4% ($p = 0.042$, Holm 0.084); on LANL the difference is 4.5% in PROACTIVE’s favour but not distinguishable from zero ($p = 0.48$). A method whose advantage over doing nothing is machine-dependent and insignificant on one of two real machines is not established.

Any runtime signal dominates. SJF_USEREST — shortest-job-first on the estimate the user actually typed — beats PROACTIVE by 20.2% on SDSC (Holm $p = 0.009$) and 15.3% on LANL. This is the same conclusion the synthetic study reached, now on real workloads with real estimate error.

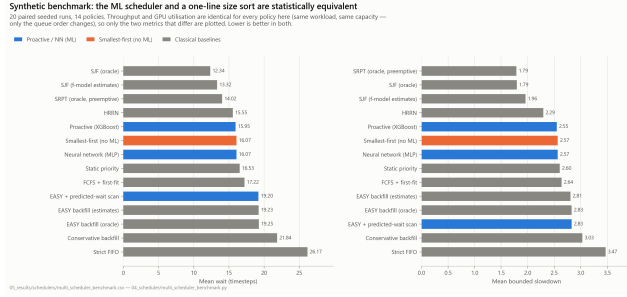


Figure 3: Fourteen schedulers on the synthetic substrate, 20 paired runs. What to look for: the NN and SMALLEST bars are not merely close but the same height — both sit at 16.074545 ts mean wait, bit-identical on every metric of all 20 runs — and PROACTIVE at 15.9477 ts is a hair’s breadth from them, inside the ± 1.59 ts equivalence margin. The ML-free sort is not a weak baseline the model narrowly beats; it is where two different model classes land. Source: 05_results/schedulers/scheduler_comparison.png, plotted from 05_results/schedulers/multi_scheduler_benchmark.csv and 05_results/schedulers/multi_scheduler_runs.csv, produced by python 04.scheduler/multi_scheduler_benchmark.py (step 7 of run_all_experiments.sh).

ML adds nothing on top of the estimate. PROACTIVE_EST, which is given the user estimate as a feature and can therefore in principle learn everything SJF exploits *plus* the cluster state, does not beat SJF_USEREST on either machine (+7.9% worse on SDSC, +17.5% on LANL). The ML pipeline’s ceiling here is the sort it is trying to learn.

Tails and fairness. Bounded slowdown separates the policies far more sharply than mean wait, and in the same direction: PROACTIVE sits at 18.5 (SDSC) where SJF_USEREST reaches 14.7 and SRPT_ORACLE 1.1. The reordering policies all buy mean wait with equity (Gini 0.88–0.89 versus 0.83 for FCFS and 0.45 for strict FIFO), reproducing the trade-off the earlier versions of this work reported.

Policy	SDSC SP2		LANL CM-5	
	wait	bsld	wait	bsld
SRPT_ORACLE	60.1	1.1	14.6	1.1
SJF_ORACLE	110.8	13.2	23.0	2.8
SJF_USEREST	115.8	14.7	31.5	4.5
HRRN_USEREST	126.3	16.8	29.5	5.0
PROACTIVE_EST	125.0	14.7	37.0	5.8
SMALLEST_FIRST	145.0	19.0	42.5	6.6
PROACTIVE	145.0	18.5	37.2	6.1
FCFS	174.6	34.7	38.8	8.2
EASY_ORACLE	183.7	35.7	46.0	8.5
EASY_USEREST	195.1	42.5	80.2	20.6
CONS_BF_USEREST	207.5	40.5	113.6	28.4
FIFO_STRICT	755.1	281.5	205.7	71.4

Table 3: Mean wait (minutes) and mean bounded slowdown, averaged over 20 paired 7-day windows per trace at offered load ≈ 0.70 . Source: 05_results/trace_schedulers/trace_scheduler_summary.csv, produced by python 04.scheduler/trace_driven_benchmark.py (step 10 of run_all_experiments.sh).

7 Real Estimate Error Versus the f -Model

Because SWF records the user’s requested time, the traces let us measure estimate error instead of simulating it. Table 4 contrasts the two machines with the f -model at $C = 5$ that our synthetic study (and much of the backfill literature) assumes.

	SDSC	LANL	f -model
Estimate present	99.9%	90.8%	—
Median $\hat{r}/r$	6.91	1.51	3.00
Mean $\hat{r}/r$	61.4	23.8	3.00
Within 2×	24.0%	41.7%	—
Under-estimates	0.1%	36.3%	0%

Table 4: Real user runtime estimates versus the standard f -model. Source: 05_results/trace_schedulers/trace_estimate_quality.csv, produced by python 04.scheduler/trace_driven_benchmark.py (step 10 of run_all_experiments.sh).

The two machines occupy opposite regimes and neither is the f -model. SDSC is extreme over-estimation (median 6.9 $\times$, nearly no under-estimates); LANL is roughly calibrated at the median but with a third of jobs under-estimated. The

On real workloads the ML scheduler lands where an ML-free size sort lands

Twelve policies replayed through two Parallel Workloads Archive traces, 20 paired 7-day windows each at offered load ≈ 0.70 , using the real user runtime estimates the traces record. Lower is better in both metrics.

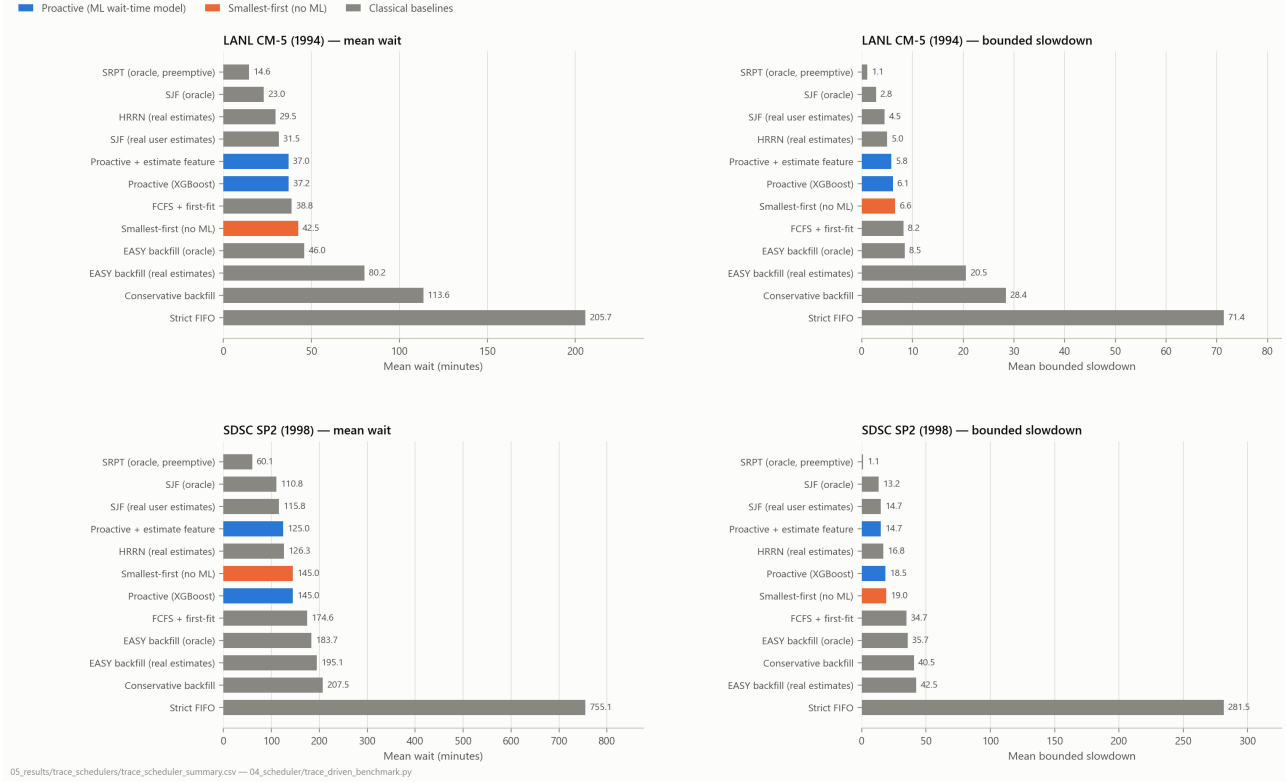


Figure 4: Twelve policies replayed through two real traces. What to look for: the PROACTIVE (ML) and SMALLEST_FIRST (ML-free) bars sit next to each other at the same height on SDSC (145.0 versus 145.0 min mean wait), which is the equivalence of §5 seen directly; and every classical policy holding a runtime estimate — the SJF, HRRN and oracle bars — finishes to the left of the ML bar on both machines. Note also how much further apart the bounded-slowdown panel spreads the same policies than the mean-wait panel does. Source: 05_results/trace_schedulers/trace_scheduler_comparison.png, plotted from 05_results/trace_schedulers/trace_scheduler_summary.csv, produced by python 04.scheduler/trace-driven_benchmark.py (step 10 of run_all_experiments.sh).

f -model, being $\hat{r} = r \cdot U(1, C)$, cannot produce an under-estimate at all.

This is not a cosmetic difference. Our synthetic study concluded that EASY is “nearly insensitive to estimate quality” (mean wait 19.0–19.8 ts across perfect, f -model, and modal estimates). On the traces, EASY with real estimates costs +6.2% mean wait versus perfect estimates on SDSC — but +74% on LANL (paired t $p = 0.025$ raw but 0.23 after Holm; the Wilcoxon signed-rank test over the same 20 windows gives Holm $p = 2.1 \times 10^{-5}$, so the effect survives correction under the rank test and not under the t -test — the wait distribution’s tail is what carries it). Under-

estimates break the mechanism EASY depends on: a job that claims it will finish before the reservation and does not *does* delay the head, so the guarantee the reservation is supposed to provide fails exactly when it matters. An evaluation using only over-estimate noise cannot observe this. We therefore think the f -model should not be used as the sole estimate-error model in backfill studies.

8 What Would Make Learning Matter

The degeneracy is a property of the feature set, not of XGBoost. A wait-time model can only pro-

duce a non-degenerate ranking if its features distinguish co-queued jobs by something other than requested size. Concretely, the feature vector must contain at least one per-job attribute that is (a) not a function of size given the state, and (b) predictive of wait. Candidates: a runtime estimate (which is precisely what SJF uses, and which §6 shows the model then merely reproduces); job-level history such as the submitting user’s recent runtimes; queue/partition identity; dependency structure; or interaction terms between a per-job attribute and the state (e.g. predicted fit against the future capacity profile rather than the current one).

More broadly, we suggest two evaluation practices. First, **report the ML-free control that the model’s own feature set implies**: if only the size feature varies across the queue, the control is sorting by size, and FIFO is not an adequate comparison. Second, **use equivalence tests** when arguing that a component matters — had we only run difference tests, the Holm-adjusted $p = 0.17$ comparison between the size sort and the XGBoost pipeline would have been reported as “no significant difference” and quietly dropped, rather than as the finding it is.

9 Threats to Validity

Reproducibility, and what is platform-dependent. Every artefact behind this paper is regenerated by a single committed pipeline, and two verifiers check it: one re-runs the pipeline in a scratch copy and diffs every artefact digit for digit, the other asserts the claims themselves. The distinction is not cosmetic. XGBoost’s histogram construction reduces floating point in parallel, so the fitted model depends on the thread count and library build of the machine that trained it; the model orders dispatch decisions, so a different machine visits slightly different instants. A second platform (a 2-vCPU Linux runner) therefore records 45,268 dispatch instants where the reference platform records 45,432 — and *zero* counterexamples on both. Instant counts in this paper should be read as reference-platform figures. The degeneracy itself is not a platform-dependent quantity, and Proposition 1 says why: it con-

strains the feature map, not the arithmetic, so any score computed from per-job inputs that are functions of size given the state is such a function in any float order. We regard the disagreement in the counts, with agreement in the result, as corroboration rather than as a defect.

Simulator fidelity, and the measured gap.

Our trace simulator models *capacity only*. It omits node-level placement and site policy (partitions, per-user limits, administrative holds), none of which SWF records, and above all it does not model the real scheduler’s policy: it replays the real arrivals, sizes, runtimes and estimates through our own FCFS, not through the queue the site actually ran. Averaged over the 20 windows of each trace, simulated FCFS mean wait is 38.82 min on LANL against 33.25 min recorded — close, and slightly *pessimistic* — but 174.62 min on SDSC against 630.92 min recorded. We state the SDSC gap plainly: on that trace the simulator is optimistic by more than a factor of three. (The recorded SDSC *median* is 19.32 min, so the recorded mean is carried by an extreme tail that a capacity-only model has no mechanism to reproduce; that explains the gap but does not close it.) SDSC results should therefore be read as “a correctly-loaded 128-processor machine driven by the real arrival, size, runtime and estimate distributions”, not as a replay of SDSC’s actual queue, and any absolute SDSC wait figure in this paper should be discounted accordingly. What survives the gap is the *relative* comparison, since all twelve policies run in the same simulator on the same windows, and the degeneracy result, which is a property of the feature vector rather than of the simulator and holds in all three substrates.

Synthetic workload realism. One of our three settings is a synthetic generator, and it is not a validated workload model: 8 nodes $\times$ 4 GPUs, uniform arrivals, sizes $U\{1, 8\}$, runtimes $U\{5, 20\}$, chosen for continuity with earlier versions of this work rather than for fidelity to any machine. Lublin and Feitelson [26], Downey and Feitelson [27], and Feitelson and Shmueli [28] describe what a principled generator would look like, and ours is none of those. Of the 45,432 dispatch

instants behind the zero-counterexample count, 3,646 come from this generator and 41,786 from the two real traces; the synthetic arm is thus the smaller share, but it is not negligible. It is also the *weaker* arm in a second respect: the Phase D adversarial attacks of §9 were run against the two traces only, so the synthetic setting has been measured but not attacked. Readers should treat the trace evidence as primary and the synthetic evidence as corroborating.

Statistical power, and one test that cannot be settled. Twenty windows per trace gives limited power; after Holm correction across the family of eleven comparisons many individually plausible differences are not significant, particularly on LANL. The LANL mean-wait equivalence test between SMALLEST_FIRST and PROACTIVE is the sharpest case, and it is worse than merely underpowered: its achieved power is 0.47%, and because the observed difference (320.02 s) *exceeds* the equivalence margin (222.93 s), TOST power falls toward zero as n grows — *no* sample size can certify equivalence there at a 10% margin. Establishing a *difference* instead would need 29 windows, or 50 after Holm correction over the family of eleven, and LANL supplies only 28 disjoint 7-day windows. The LANL row is therefore **inconclusive**, not “different”, and it cannot be settled on that trace at all; settling it requires a different or longer trace. We say this rather than reporting the failed equivalence test as evidence of a difference, which is the error the paper’s own methodology section warns against. Elsewhere we lean on effect sizes, on the consistency of the ordering across both machines, and — for the sameness claims — on equivalence tests rather than on failures to reject. Source: 05_results/trace_schedulers/tost_power.csv, produced by python 04_scheduler/tost_power.py (step 11 of run_all_experiments.sh).

Which model split the accuracy number comes from. The wait model’s headline accuracy in this paper is the run-wise grouped split: $R^2 = 0.810910 \pm 0.020672$ and MAE 4.8970 over five GroupKFold folds on the run identifier. Earlier versions of this work quoted a

uniformly random row-level split, $R^2 = 0.836840$ and MAE 4.6935, which is optimistic: the 2,200 rows are 20 simulation runs of 110 jobs each, so a random split puts rows of the same run — and adjacent instants of it — on both sides. Under a within-run *chronological* split, which is what deployment actually means (train on a run’s earlier arrivals, predict its later ones), R^2 falls to 0.725147 and MAE rises to 7.2403. We report the run-wise figure as the headline and the chronological figure as the deployment bound. None of this touches the degeneracy result: that result is a statement about the *functional form* of the score — that it factors through requested size — and it holds for a perfect model exactly as it holds for a poor one. This caveat bounds the deployment story, not the contribution. Source: 05_results/models/evaluation_splits.csv, produced by python 03_models/evaluate_splits.py (step 23 of run_all_experiments.sh).

Covariate shift in the retrained model. The per-trace model is trained on features reconstructed from the *recorded* schedule, produced by the site’s real scheduler. When PROACTIVE runs it induces different states than those it was trained on. This is inherent to the approach rather than an artefact of our setup, but it means the model is evaluated slightly off-distribution even after re-training.

Vintage of the traces. LANL CM-5 (1994) and SDSC SP2 (1998) are batch HPC machines. Their users supply runtime estimates, which is exactly the regime in which the ML scheduler has no niche. Modern deep-learning clusters, where job durations are often genuinely unknown at submit time, are the setting in which a zero-runtime-information policy could still be worth something; validating there is the clearest next step, and we make no claim about it here.

The scope of the negative result. This is a negative result about *one feature family*, not about ML scheduling in general, and we want the boundary to be unmistakable. The claim is: *a score computed at the dispatch instant, over features that consist of cluster state plus per-job attributes that*

factor through requested size given that state, cannot rank two co-queued jobs of equal size. Everything outside that description is untouched.

We tested the boundary in both directions. In the converse direction, adding per-job attributes that are genuinely not functions of size given the state — the user’s runtime estimate, causal per-user history, queue identity, user identity — breaks the degeneracy immediately: counterexample counts rise from 0 to between 6,194 and 10,718 on SDSC and from 0 to between 13,801 and 28,497 on LANL, across five such attributes each of which breaks it on its own. Breaking the degeneracy is not the same as scheduling better, however: 0 of the 12 augmented variants beat SJF_USEREST, and the best of them (LANL with all attributes added) is 1.48% *slower* than that plain sort and TOST-equivalent to it ($p = 0.0014$).

In the adversarial direction, we attacked the claim itself. A pairwise learning-to-rank objective (`rank:pairwise`), a listwise one (`rank:ndcg`), monotone score transforms (`log1p` and a sigmoid), and a 5-tick lagged history window all yield *zero* counterexamples: the degeneracy is a property of the feature map, and changing the loss or monotonically reshaping the score does not touch it. One attack does break the claim, and it breaks it by violating the premise rather than by evading the argument: scoring a job when it *enters* the queue and caching that score gives 20,482 (SDSC) and 37,741 (LANL) counterexamples, because two equally-sized jobs then carry scores computed against *different* cluster states. This is the shared-state assumption failing by construction, and it is why we scope the claim to a score computed at the dispatch instant. It also costs: enqueue-time caching is worse on LANL (+11.74% mean wait and 27% worse bounded slowdown) and, while nominally 1.64% better on SDSC mean wait, raises SDSC median wait from 76.0 to 105.4 s and bounded slowdown from 18.49 to 21.56. Source: `05_results/degeneracy/non_degeneracy_sweep.csv` and `05_results/degeneracy/non_degeneracy_utility.csv`, produced by python `04_scheduler/non_degeneracy_sweep.py` (step 12 of `run_all_experiments.sh`); and `05_results/degeneracy/robustness_attacks.csv` and `05_results/degeneracy/robustness_attack_utility.csv`, produced by python `04_scheduler/robustness_attacks.py` (step 13).

produced by python `04_scheduler/robustness_attacks.py` (step 13).

10 Conclusion

We set out to find where an ML wait-time scheduler adds value and found that, as constructed, it cannot add value to queue ordering: its score is a function of requested size alone, verified over 45,432 dispatch instants without a single counterexample. A one-line sort is statistically equivalent to it on two of three substrates, an MLP over the same features reproduces the sort exactly, its synthetic advantage over FCFS does not replicate on real traces, and any classical policy holding a runtime estimate beats it on both real machines. Alongside this we report that real user-estimate error damages backfill far more than the standard f -model predicts, because a third of LANL’s estimates are under-estimates that the f -model cannot generate.

We report these as negative results because they are the results. The constructive content is the non-degeneracy condition of §8 and the methodological point that an ML systems component should be compared against the ML-free control its own feature set implies — with an equivalence test, not a failed difference test.

Artifact Availability

Code, both SWF traces, the seeded pipeline (`run_all_experiments.sh`), and every CSV behind the tables are in the repository. `04_scheduler/ranking_degeneracy.py` reproduces §4, `trace_driven_benchmark.py` reproduces §6–7, and `multi_scheduler_benchmark.py` reproduces §5.

References

- [1] D. G. Feitelson, *Workload Modeling for Job Scheduling in Parallel Computers*, EWOPP 1997.
- [2] A. Yoo et al., *SLURM: Simple Linux Utility for Resource Management*, JSSPP 2003.

- [3] R. Jain, D.-M. Chiu, and W. Hawe, *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems*, DEC Research Report TR-301, 1984.
- [4] D. G. Feitelson, *Metrics for Parallel Job Scheduling and Their Convergence*, JSSPP 2001.
- [5] D. G. Feitelson, D. Tsafir, and D. Krakov, *Experience with Using the Parallel Workloads Archive*, J. Parallel Distrib. Comput. 74(10):2967–2982, 2014. <https://www.cs.huji.ac.il/labs/parallel/workload/>
- [6] A. W. Mu’alem and D. G. Feitelson, *Utilization, Predictability, Workloads, and User Runtime Estimates in Scheduling the IBM SP2 with Backfilling*, IEEE TPDS 12(6), 2001.
- [7] D. Tsafir, Y. Etsion, and D. G. Feitelson, *Modeling User Runtime Estimates*, JSSPP 2005.
- [8] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, *Resource Management with Deep Reinforcement Learning*, HotNets 2016.
- [9] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, *Learning Scheduling Algorithms for Data Processing Clusters*, SIGCOMM 2019.
- [10] D. Tsafir, Y. Etsion, and D. G. Feitelson, *Backfilling Using System-Generated Predictions Rather Than User Runtime Estimates*, IEEE TPDS 18(6):789–803, 2007.
- [11] A. Nissimov and D. G. Feitelson, *Probabilistic Backfilling*, JSSPP 2007.
- [12] E. Shmueli and D. G. Feitelson, *Backfilling with Lookahead to Optimize the Packing of Parallel Jobs*, JSSPP 2003.
- [13] E. Shmueli and D. G. Feitelson, *Backfilling with System-Generated Job Runtime Estimates*, IPDPS 2005.
- [14] D. Talby and D. G. Feitelson, *Supporting Priorities and Improving Utilization of the IBM SP Scheduler Using Slack-Based Backfilling*, IPPS/SPDP 1999.
- [15] D. G. Feitelson, *Experimental Analysis of the Root Causes of Performance Evaluation Results: A Backfilling Case Study*, IEEE TPDS 16(2):175–182, 2005.
- [16] T. H. L. Hai, K. N. Duy, T. N. Manh, D. M. Hoang, and N. Thoai, *Deviation Backfilling: A Robust Backfilling Scheme for Improving the Efficiency of Job Scheduling on High Performance Computing Systems*, IEEE, 2023.
- [17] T. N. Minh and L. Wolters, *Using Historical Data to Predict Application Runtimes on Backfilling Parallel Systems*, IEEE, 2010.
- [18] Y. Yuan, Y. Wu, W. Zheng, and K. Li, *Guarantee Strict Fairness and Utilize Prediction Better in Parallel Job Scheduling*, IEEE TPDS, 2014.
- [19] J.-W. Park, *Queue Waiting Time Prediction for Large-scale High-performance Computing System*, IEEE, 2019.
- [20] J.-W. Park and T. Hong, *Improving Runtime Prediction Performance Based on Application Type of Jobs on Large-scale Cluster System*, IEEE, 2025.
- [21] A. Lovell, P. Wisniewski, S. Rodenbeck, et al., *A Hierarchical Deep Learning Approach for Predicting Job Queue Times in HPC Systems*, IEEE, 2024.
- [22] S. Ramachandran, M. L. Jayalal, et al., *Combining Machine Learning and Metaheuristic Algorithms for Predicting Waiting Time of High Performance Computing Jobs*, IEEE, 2024.
- [23] L. Barsanti and A. C. Sodan, *Adaptive Job Scheduling via Predictive Job Resource Allocation*, JSSPP 2006.
- [24] H. Wang, Z. Liu, and H. Shen, *Machine Learning Feature Based Job Scheduling for Distributed Machine Learning Clusters*, IEEE/ACM Trans. Networking 31(1):58–72, 2023.

- [25] M. Hovestadt, O. Kao, A. Keller, and A. Streit, *Scheduling in HPC Resource Management Systems: Queuing vs. Planning*, JSSPP 2003.
- [26] U. Lublin and D. G. Feitelson, *The Workload on Parallel Supercomputers: Modeling the Characteristics of Rigid Jobs*, J. Parallel Distrib. Comput. 63(11):1105–1122, 2003.
- [27] A. B. Downey and D. G. Feitelson, *The Elusive Goal of Workload Characterization*, ACM SIGMETRICS Perf. Eval. Rev. 26(4):14–29, 1999.
- [28] D. G. Feitelson and E. Shmueli, *A Case for Conservative Workload Modeling: Parallel Job Scheduling with Daily Cycles of Activity*, MASCOTS 2009.
- [29] D. Zhang, M. S. Raj, B. Xie, S. Di, and D. Dai, *Cross-System Analysis of Job Characterization and Scheduling in Large-Scale Computing Clusters*, IEEE, 2024.
- [30] C. Burges, T. Shaked, E. Renshaw, A. Lazier, M. Deeds, N. Hamilton, and G. Hullender, *Learning to Rank Using Gradient Descent*, ICML 2005.
- [31] T.-Y. Liu, *Learning to Rank for Information Retrieval*, Foundations and Trends in Information Retrieval 3(3):225–331, 2009.
- [32] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, *The Case for Learned Index Structures*, SIGMOD 2018.
- [33] R. Marcus, A. Kipf, A. van Renen, M. Stoian, S. Misra, A. Kemper, T. Neumann, and T. Kraska, *Benchmarking Learned Indexes*, Proc. VLDB Endowment 14(1):1–13, 2020.
- [34] T. G. Armstrong, A. Moffat, W. Webber, and J. Zobel, *Improvements That Don't Add Up: Ad-Hoc Retrieval Results Since 1998*, CIKM 2009.
- [35] J. Lin, *The Neural Hype and Comparisons Against Weak Baselines*, ACM SIGIR Forum 52(2):40–51, 2018.
- [36] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, *Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches*, RecSys 2019.
- [37] K. Musgrave, S. Belongie, and S.-N. Lim, *A Metric Learning Reality Check*, ECCV 2020.
- [38] D. Sculley, J. Snoek, A. Wiltschko, and A. Rahimi, *Winner's Curse? On Pace, Progress, and Empirical Rigor*, ICLR 2018 Workshop Track.
- [39] C. Collberg and T. A. Proebsting, *Repeatability in Computer Systems Research*, Communications of the ACM 59(3):62–69, 2016.
- [40] A. V. Goponenko, K. Lamar, C. Peterson, B. A. Allan, J. M. Brandt, and D. Dechev, *Metrics for Packing Efficiency and Fairness of HPC Cluster Batch Job Scheduling*, IEEE, 2022.